

图分类号: TP18

学校代码: 10036

UDC 分类号: 004.8

密 级: 非密



对外经济贸易大学

University of International Business and Economics

本科毕业论文

基于视觉感知的 网页自动导航模型优化研究

论文作者: 李念一

学 号: 202293012

培养单位: 人工智能与数据科学学院

专业名称: 人工智能

指导教师: 许萌

二〇二六年三月

图分类号: TP18

学校代码: 10036

UDC 分类号: 004.8

密 级: 非密



对外经济贸易大学

University of International Business and Economics

Graduation Thesis

Optimization of Automatic Web Navigation Models Based on Visual Perception

Student Name : Li Nianyi

Student ID No.: 202293012

**Department/School : School of Artificial
Intelligent and Data Science**

Major Field: Artificial Intelligence

Advisor: XuMeng

2026-3

毕业论文（设计）原创性声明

本人郑重声明：所呈交的毕业论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不含任何其他个人或集体已经发表或撰写过的作品成果。对本文所涉及的研究工作做出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律责任由本人承担。

特此声明。

毕业论文作者签名：

毕业论文（设计）版权使用授权书

本人完全了解对外经济贸易大学关于收集、保存、使用毕业论文的规定，同意如下各项内容：按照学校要求提交毕业论文的印刷本和电子版本；学校有权保存毕业论文的印刷本和电子版，并采用影印、缩印、扫描、数字化或其它手段保存论文；学校有权提供目录检索以及提供本毕业论文全文或部分的阅览服务；学校有权按照有关规定向国家有关部门或者机构送交论文；学校可以采用影印、缩印或者其它方式合理使用毕业论文，或将毕业论文的内容编入相关数据库供检索；保密的毕业论文在解密后遵守此规定。

毕业论文作者签名：

年 月 日

导师签名：

年 月 日

对外经济贸易大学《人工智能技术使用声明》

姓名：李念一	学号：202293012
学院：人工智能与数据科学学院	专业：人工智能
类型（勾选）：博士研究生 ☐ 硕士研究生（含同等学力） ☐ 留学生 ☐ 本科生 ☒	
论文题目：基于视觉感知的网页自动导航模型优化研究	
人工智能使用情况说明：根据《对外经济贸易大学在毕业论文和毕业论文（设计）中规范使用人工智能技术的指导意见（试行）》（贸大研字（2025）16 号），学生可借助人工智能技术跟踪研究动态、收集整理文献资料，生成模拟仿真数据、对原始数据进行统计分析，进行辅助制图，进行文字校对、论文格式校验等。请依次介绍在毕业论文和毕业论文（设计）中已使用的人工智能情况，包括工具名称、版本、日期、使用过程、是否标注等。如未使用，请注明“本人在毕业论文（设计）写作过程中未使用过人工智能技术”。	
声明承诺	
本人在使用人工智能技术过程中，能按照学校要求在成果相应位置标注，在注释、致谢、参考文献或附录等部分声明工具的名称、版本、日期及使用过程。自觉接受学校对本人使用人工智能工具情况的监督与检查。 本人签字： 日 期：	

摘 要	3
Abstract	4
第 1 章 引言	6
1.1 研究背景与意义	6
1.2 国内外研究现状	7
1.3 研究问题与挑战	8
1.4 本文主要工作	9
第 2 章 WebVLN 任务与数据集	10
2.1 WebVLN 任务定义	10
2.2 WebVLN 数据集结构	11
2.3 环境划分与评测设置	12
2.4 候选动作机制分析	13
2.5 数据加载与预处理细节	14
2.6 数据集统计分析与问题归纳	15
2.7 问题分析与研究动机	17
第 3 章 WebVLN 模型结构	17
3.1 模型整体结构概述	17
3.2 语言编码模块	19
3.3 候选动作编码模块	20
3.4 状态与历史信息建模	21
3.5 跨模态融合机制	22
3.6 动作预测与决策过程	22
3.7 问答解码模块	23
3.8 模型训练机制	24
3.9 训练流程与验证策略详解	25
3.10 模型存在的问题分析	26

第 4 章 基于大模型的候选动作筛选方法	27
4.1 方法总体设计	27
4.2 候选动作的语义化表示重构	28
4.3 基于大模型的相关性排序模型	29
4.4 两阶段筛选机制设计	30
4.5 Top-k 参数分析与评估指标	31
4.6 筛选模块与原始模型的衔接	32
第 5 章 实验设计与结果分析	33
5.1 实验环境、配置与评测指标	33
5.2 基线模型复现：逻辑与过程验证	35
5.3 候选筛选方法整体效果对比	37
5.4 候选空间压缩与关键动作保留分析	39
5.5 两阶段筛选策略的有效性验证	40
5.6 典型案例分析与局限性讨论	41
5.7 实验结果总结	43
第 6 章 结论与展望	44
6.1 研究结论	44
6.2 研究局限	45
6.3 未来展望	47
6.4 结束语	48
参考文献	48
致谢	50

摘要

随着互联网信息的爆炸式增长，用户通常需要通过多步点击与页面跳转来完成复杂的在线任务。基于视觉与语言的网页自动导航任务旨在构建能够理解自然语言指令并在网页环境中执行多步操作的智能体。然而，现有模型在面对网页中数量庞大且质量参差不齐的候选可点击元素时，决策过程易受噪声干扰，导致导航性能受限。

针对这一问题，本文在成功复现 WebVLN 基线模型的基础上，提出了一种基于大语言模型的候选动作筛选优化方法。首先对原模型进行了完整的复现验证：在 PyTorch 平台上使用官方数据集和超参数配置，从头训练 200,000 次迭代，得到的模型在验证集上的成功率为 38.35%，路径效率为 38.30%，在测试集上的成功率为 34.21%，路径效率为 34.13%，与原论文报告结果高度一致，确认了实验环境的正确性。通过轨迹回放分析发现，模型在多步决策中倾向于点击导航栏或侧边栏的高频通用链接而非与指令语义最相关的内容链接，验证了候选噪声是制约性能的主要障碍。

语言模型对候选动作与导航指令的相关性进行评估与排序，并通过 Top- k 策略筛选出高质量子集以压缩决策空间。进一步设计了一种规则粗筛与语义精排相结合的两阶段筛选机制，以平衡计算开销与筛选精度。

本文的研究验证了将大语言模型作为语义过滤器来优化多模态决策模型输入空间的有效性，为网页自动导航任务的输入质量优化提供了一种可行的技术路径。

关键词：网页自动导航；视觉与语言导航；候选动作筛选；大语言模型；多模态决策

Abstract

With the explosive growth of internet information, users often need to perform multiple clicks and page jumps to complete complex online tasks. The Vision-and-Language Navigation on websites task aims to build intelligent agents capable of understanding natural language instructions and executing multi-step operations within web environments. However, existing models face significant challenges due to the large scale and variable quality of candidate clickable elements on web pages, making the decision-making process susceptible to noise interference.

To address this issue, this paper proposes a candidate action screening optimization method based on Large Language Models, built upon the successful reproduction of the baseline WebVLN model. The baseline model is first fully reproduced on the PyTorch platform using the official dataset and hyperparameters. After training for 200,000 iterations, the reproduced model achieves a success rate of 38.35% and an SPL of 38.30% on the validation set, and a success rate of 34.21% and an SPL of 34.13% on the test set, which are highly consistent with the original paper’s reported results, thereby confirming the correctness of the experimental setup. Trajectory analysis reveals that the model tends to click on high-frequency generic links in navigation bars or sidebars rather

than content links semantically most relevant to the instruction, verifying that candidate noise is a major obstacle to performance.

Based on this reproduction, the proposed method first reconstructs the structured candidate features into semantic text descriptions, then leverages the powerful semantic understanding of LLMs to evaluate and rank the relevance between each candidate and the navigation instruction, and finally selects a high-quality subset via a Top-k strategy to compress the decision space. Furthermore, a two-stage screening mechanism combining rule-based filtering and semantic ranking is designed to balance computational cost and accuracy. This work demonstrates the effectiveness of using LLMs as semantic filters to optimize the input space of multi-modal decision-making models, providing a feasible technical path for input quality optimization in automatic web navigation tasks.

Keywords: Web Navigation; Vision-and-Language Navigation; Candidate Action Screening; Large Language Model; Multimodal Decision Making

第 1 章 引言

1.1 研究背景与意义

随着互联网信息规模的持续增长，网页已经成为人类获取信息与完成在线任务的重要载体。在实际应用场景中，用户往往需要通过多次点击与页面跳转，才能完成复杂的信息检索或操作任务，例如查找商品信息、浏览技术文档或完成在线服务流程。然而，传统的信息检索方法主要依赖关键词匹配或单步查询，难以有效处理需要多步骤决策的复杂任务。在此背景下，基于人工智能的自动网页导航技术逐渐成为研究热点。该类技术旨在构建智能体，使其能够理解用户给出的自然语言指令，并在网页环境中通过多步交互完成目标任务。这一问题不仅涉及自然语言理解，还需要结合视觉感知与决策规划能力，因此具有显著的跨模态特征和较高的研究价值。

近年来，视觉与语言导航任务在室内环境中取得了重要进展。早期研究主要集中于模拟三维室内环境，例如 Room-to-Room 任务（一种要求智能体根据自然语言指令在真实室内场景中从起点房间导航至终点房间的视觉语言导航任务）。相关研究通过构建三维空间环境，使智能体根据语言指令完成路径规划与导航操作。然而，与室内导航场景相比，网页环境具有结构复杂、元素类型多样、语义信息丰富等特点，使得导航问题更加具有挑战性。例如，室内导航中候选路径通常局限于物理可通行区域，而网页环境中每个页面可包含数十个可点击超链接、按钮或输入框，决策空间呈指数级增长。因此，将视觉语言导航方法拓展至网页环境，并研究其在复杂信息空间中的表现，具有重要的理论意义与应用价值。

Web Vision and Language Navigation（网页导航模型技术）任务^[8]正是在上述背景下提出的。该任务要求模型在网页环境中，根据自然语言指令逐步选择可点击元素，最终完成导航与问答任务。相比传统导航任务，WebVLN 不仅需要处理视觉信息，还需要理解网页中的结构化元素与语义关系，从而对模型提出了更

高要求。因此，对 WebVLN 模型进行深入研究改进，对于推动多模态智能体的发展具有重要意义。

此外，随着大语言模型技术的飞速发展，其在语义理解、推理与规划方面的能力被广泛认可。将大语言模型应用于 WebVLN 任务中，用以辅助候选动作理解、路径规划或错误恢复，已成为当前研究的前沿方向。代表性工作包括 WebGUM^[14]、AutoWebGLM^[15]和 WebVoyager^[16]等。然而，现有工作多侧重于将大语言模型作为直接的决策器或规划器，忽视了其作为输入空间“语义过滤器”的潜力。本文聚焦于利用大语言模型优化 WebVLN 模型的输入质量，在不改变原有决策模型结构的前提下提升导航性能，这一思路对实际系统部署具有重要的参考价值。

1.2 国内外研究现状

在视觉语言导航领域，早期研究主要集中于模拟环境或真实室内场景。例如，Room- to- Room 任务^[1]通过构建三维室内空间，使智能体能够根据语言指令完成路径选择与导航操作。这类研究推动了多模态学习的发展，并提出了多种基于注意力机制与序列建模的方法。随着深度学习技术的发展，基于 Transformer 结构的模型逐渐成为主流。尤其是 BERT^[6]等预训练语言模型的提出，使得语言表示能力显著提升。在此基础上，研究者进一步提出了多模态融合模型，如 VLN- BERT，通过对语言特征与视觉特征进行联合建模，有效提升了导航任务性能。此外，LXMERT^[2]、ViLBERT^[3]、UNITER^[4]等工作在多模态预训练领域探索了图像与文本的对齐方式，为 WebVLN 中视觉- 语言联合编码提供了技术基础。

在网页导航方向，相关研究逐渐从简单的规则方法转向基于学习的方法。早期基于 DOM 树解析和 XPath 定位的自动化工具虽然能完成固定流程任务，但缺乏泛化性和语义理解能力。WebVLN 任务通过构建网页环境与导航数据集，使模型能够在复杂网页结构中进行决策。然而，与室内导航相比，网页导航面临更复杂的候选动作空间与更丰富的语义信息，这使得模型在决策过程中容易受到噪声干扰。例如，一个典型电商页面可能包含多达上百个可点击元素，其中仅有 5 到 10 个

与当前导航指令高度相关，其余如“隐私政策”“广告横幅”“站内搜索”等链接构成了巨大的背景噪声。

近年来，大规模预训练模型在自然语言理解与推理任务中表现出显著优势。这类模型具备较强的语义建模能力，为解决复杂决策问题提供了新的思路。然而，目前大模型在网页导航中的应用仍处于探索阶段，尤其是在如何与现有导航模型有效结合方面，尚缺乏系统性研究。一些初步工作如 NavGPT 尝试使用大语言模型进行导航推理[20]，但存在计算成本高、实时性差等问题。因此，将大语言模型引入 WebVLN 任务，并用于优化导航过程中的关键环节——候选动作筛选与输入空间精炼，具有较大的研究潜力。

1.3 研究问题与挑战

尽管现有 WebVLN 模型在网页导航任务中取得了一定进展，但仍存在若干关键问题亟需解决。

1. 候选动作空间规模较大。在每一步导航过程中，模型需要在当前页面的所有可点击元素中进行选择，而这些候选动作往往包含大量与任务无关的信息，例如导航栏链接、广告区域或页脚内容。这种冗余候选会显著增加模型的决策难度。通过实际统计发现，WebVLN 数据集中平均每步候选数量约为 45 个，部分复杂页面甚至超过 100 个，导致模型需要在庞大的动作空间中执行分类。

2. 候选动作质量参差不齐。网页中的可点击元素具有不同的语义重要性，其中只有少数元素与当前任务目标直接相关，而其余元素则构成噪声干扰。这种情况会导致模型在训练与推理过程中难以有效聚焦关键动作。尤其是在使用监督学习训练时，大量负样本会使模型倾向于记忆页面中高频出现的通用链接，例如“首页”“关于我们”，而非学习真正的语义匹配能力。

3. 决策过程复杂度较高。WebVLN 模型需要同时完成语言理解、视觉感知与动作选择任务，在候选空间较大的情况下，模型容易出现错误决策或路径冗余，从而影响整体性能。此外，模型在未见网页环境中的泛化能力仍有待提升。由于网

页结构差异较大，模型在训练环境中学习到的模式难以直接迁移至新的网页环境，这进一步增加了任务难度。

4. 跨模态信息融合不充分。现有模型在处理候选动作时，将文本特征、视觉特征和位置信息进行简单拼接后输入 Transformer。然而，网页中元素的语义不仅取决于其自身属性，还依赖于周围上下文和页面布局。自注意力机制虽然能够建模全局依赖，但当噪声候选占绝大多数时，注意力权重容易被无关元素分散。

综上所述，如何在不显著增加模型复杂度的前提下，有效降低候选动作空间中的噪声干扰，并提升模型决策质量，是当前 WebVLN 研究中的重要问题。

1.4 本文主要工作

针对 WebVLN 任务中候选动作空间较大、存在大量无关信息干扰等问题，本文围绕模型复现与方法改进两个方面展开研究，主要工作如下。

1. 基线模型复现与问题诊断。在 PyTorch 平台上完整复现了 WebVLN 基线模型，从头训练 200,000 次迭代后，验证集成功率为 38.35%，测试集成功率为 34.21%，与原论文结果高度一致，确认了实验环境的正确性。通过轨迹回放分析发现，模型在多步决策中倾向于点击导航栏或侧边栏的高频通用链接而非与指令语义最相关的内容链接，从而验证了候选噪声是制约性能的主要障碍。

2. 基于大语言模型的候选动作筛选方法。提出了一种可插拔的候选筛选框架，在不改变原模型主体结构的前提下，将候选动作重构为语义化文本描述，利用大语言模型对候选与指令的相关性进行排序，并通过 Top k 策略筛选出高质量子集以压缩决策空间。进一步设计了规则粗筛与语义精排相结合的两阶段筛选机制，在保证筛选精度的同时有效降低了大模型调用成本。

3. 系统性实验验证与分析。构建了多组对比实验，从候选压缩率、正确动作保留率以及导航成功率等多个维度评估了所提方法的有效性。实验结果表明，在最优参数设置下，模型在测试集上的成功率和路径效率均优于基线水平，且两阶

段筛选策略将大模型 API 调用成本降低约 50%，验证了输入空间优化对多模态决策性能的提升效果。

综上所述，本文在完成 WebVLN 模型复现的基础上，从候选动作优化的角度提出了一种基于大语言模型的改进方法，并通过系统实验验证了其有效性，为网页导航任务的研究提供了新的思路。

第 2 章 WebVLN 任务与数据集

2.1 WebVLN 任务定义

Web Vision-and-Language Navigation，简称 WebVLN，是一类典型的多模态决策任务，其目标是构建能够理解自然语言指令并在网页环境中执行多步操作的智能体。该任务最初由 Chen 等人提出^[8]。在该任务中，智能体需要根据输入的语言指令，在网页环境中逐步选择可点击元素，通过多次页面跳转，最终完成指定目标。与传统的视觉语言导航任务相比，WebVLN 任务具有更加复杂的环境结构。具体而言，网页环境不仅包含视觉信息，还包含大量结构化文本信息以及丰富的交互元素，例如超链接、按钮、输入框等。这些元素共同构成了导航过程中的候选动作集合。

在形式化定义上，WebVLN 任务可以表示为一个序列决策问题^[10]。给定初始页面状态以及自然语言指令，智能体在每一步需要从当前页面的候选动作集合中选择一个动作，并根据该动作跳转至新的页面状态。该过程不断重复，直到达到终止条件或完成任务目标。设导航过程为一个状态转移序列：

$$s_0, s_1, s_2, \dots, s_T \quad (2.1.1)$$

其中 s_0 表示初始页面状态， s_T 表示最终状态。在每一个状态 s_T 下，存在一个候选动作集合：

$$\mathcal{A}_t = \{a_t^1, a_t^2, \dots, a_t^{M_t}\} \quad (2.1.2)$$

模型需要从中选择一个动作 a_t ，并执行状态转移：

$$s_{t+1} = f(s_t, a_t) \quad (2.1.3)$$

模型的目标是学习一个策略函数，使得在整个决策序列中能够最大程度地完成导航任务，并提高成功率与路径效率。

值得注意的是，WebVLN 任务中的“成功”通常具有多重定义：既包括最终到达包含答案的页面，也包括提取出正确的答案文本。因此，评测指标涵盖了导航成功率 SR 和答案语义匹配度 WUPS，这要求模型不仅具备路径规划能力，还需具备内容理解与信息提取能力。WebVLN 候选动作构建集合如图 2.1 所示：

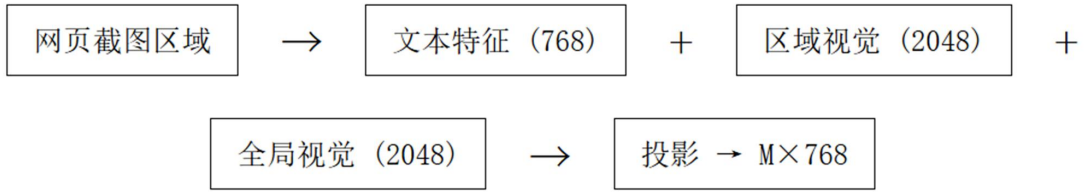


图 2.1 WebVLN 候选动作构建集合示意图

2.2 WebVLN 数据集结构

WebVLN 任务依赖于特定构建的数据集^[8]，该数据集通常由网页环境、导航指令以及对应的操作轨迹组成。每一条数据样本包含以下几个核心部分。

1. 导航指令。该部分为自然语言描述，定义了用户希望完成的任务目标，例如“找到某商品的价格信息”或“进入指定页面查看内容”。指令的语言风格接近真实用户查询，包含指代、省略和模糊表达。数据集中的每条指令由一个问题 Q 和一个辅助描述 D 构成，格式为“Q: ... D: ...”。辅助描述在问题信息不充分时提供额外的上下文，例如商品的具体特征。

2. 网页状态。网页状态包含当前页面的视觉信息与结构信息，通常以页面截图或 DOM 结构形式表示。在 WebVLN 数据集中，每个页面还提供了渲染后的截图特征，这些特征已由数据集构建者使用 ResNet152 模型预先提取为向量形式，存

储为 PyTorch 张量文件。同时，每个页面还记录了该页面中所有可点击元素的位置、文本内容以及对应的截图区域特征。

3. 操作轨迹。轨迹由一系列动作组成，每一个动作对应网页中的一个可点击元素。该轨迹表示从初始页面到目标页面的完整路径。数据集中的轨迹由众包工人按照指令实际操作生成，包含了多步点击和页面跳转。在本文复现所用的 WebVLN v1 数据集中，平均轨迹长度为 3.32 步，最短路径为 2 步，最长不超过 10 步。

4. 候选动作集合。在每一个导航步骤中，当前页面包含多个可点击元素，这些元素构成候选动作集合。模型需要从该集合中选择最合适的动作。每个候选动作被表示为三元组：元素文本或 alt 属性、该元素在截图中的局部区域特征、以及该元素对应的动作标识。值得注意的是，候选动作集合中除了正常的导航链接外，还包含一个特殊的停止动作[EOA]，用于指示模型已经到达目标页面并应停止导航。

从数据结构角度来看，WebVLN 数据集本质上是一个“指令-状态-动作”三元组序列，其中每一步都包含局部决策信息。这种结构使得模型需要同时考虑当前状态与历史信息，从而完成多步推理。

2.3 环境划分与评测设置

为了评估模型在不同场景下的泛化能力，WebVLN 任务采用基于环境的划分方式^[8]。具体而言，数据集被划分为训练集、验证集和测试集，其中不同集合中的网页环境互不重叠。这种划分方式通常被称为“环境级划分”，其核心思想是：训练集与测试集中的网页来源于不同的网站或页面结构，从而避免模型仅通过记忆页面模式完成任务。

在 WebVLN v1 数据集中，数据来自三个不同的购物网站，分别别名为 SA、HB 和 ES。划分比例如下。

1. 训练集：约占全部样本的百分之六十，共计约 8,960 条路径。

2. 验证集：约占全部样本的百分之十，共计约 1,262 条路径。

3. 测试集：约占全部样本的百分之三十，共计约 4,603 条路径。

需要特别说明的是，训练集、验证集和测试集均覆盖了上述三个网站，但各自的导航路径记录互不重叠。这一设计确保了模型在训练过程中未见过的页面结构可以作为验证和测试的素材。

此外，根据网页环境的不同，还可以进一步划分为“seen”和“unseen”两种设置。其中，“seen”表示测试环境与训练环境在一定程度上存在相似性，例如同为电商类网站但具体商品页面不同；而“unseen”则要求模型在完全未知的网页环境中进行导航，例如从电商环境迁移至学术文档环境。本文实验主要关注 seen 环境下的性能提升，同时保留了对 unseen 环境的泛化能力讨论。这种划分方式能够更真实地反映模型在实际应用中的表现，是评价 WebVLN 模型性能的重要标准。

2.4 候选动作机制分析

在 WebVLN 任务中，候选动作集合是模型进行决策的核心依据。每一步导航中，模型都需要在当前页面的候选动作集合中选择一个动作，这一过程本质上可以视为一个分类问题。候选动作通常来源于网页中的可交互元素，例如超链接、按钮或其他可点击组件。对于每一个候选动作，数据集中会提供对应的特征表示，包括视觉特征、文本特征以及位置信息等。

在本文复现过程中，对候选动作的具体处理方式如下。

1. 从数据文件中加载当前页面的所有可点击元素信息。每个元素被表示为一个字典，包含以下字段：元素类型、元素文本、截图区域坐标、对应的动作索引。

2. 对于每个元素，从其对应的预提取特征文件中读取三类特征。文本特征由 BERT 提取，维度为 768；截图区域特征由 ResNet 152 提取，维度为 2048；页面全局截图特征同样由 ResNet 152 提取，维度为 2048。

3. 将上述三类特征拼接为维度为 4864 的临时特征向量，然后通过一个可学习的线性层投影到 768 维，与语言特征的维度对齐。

4. 将所有候选动作的投影后特征堆叠为一个矩阵，形状为 $M \times 768$ ，其中 M 为当前页面的候选数量。

然而，通过对实际数据与训练过程的分析可以发现，候选动作集合存在以下问题。

1. 候选数量较多。在实际网页中，一个页面可能包含数十甚至上百个可点击元素，这导致模型在每一步需要处理较大的动作空间。在 WebVLN v1 数据集中，平均每个页面的候选数量约为 45 个，部分复杂页面的候选数量超过 100 个。例如，在 SA 网站的首页，包含顶部导航栏、侧边栏商品分类、中部轮播图链接以及页脚信息链接，总计可点击元素可达 120 余个。

2. 候选质量不均。大量候选元素与当前导航任务无关，例如广告链接或导航栏元素，这些无关信息会对模型决策造成干扰。特别是在模型训练初期，由于缺乏有效的先验筛选，大量噪声候选淹没有限的正样本信号，导致模型收敛缓慢。例如，在一个查找特定商品价格的轨迹中，正确的动作可能是点击“产品详情”链接，但页面上同时还存在“隐私政策”“关于我们”“加入购物车”等多个与当前目标无关的链接。模型需要从数十个候选中识别出正确的一个，难度较大。

3. 语义信息复杂。候选动作不仅包含文本信息，还与页面结构及上下文密切相关，这增加了模型理解难度。例如，“更多”这样的按钮文本在不同的页面上下文中可能指向产品详情、评论列表或技术支持页面，模型必须结合周围上下文才能准确判断。数据集中的候选动作仅提供了元素自身的文本描述，并未显式提供上下文信息，这要求模型在跨模态融合过程中自行捕捉上下文关联。

因此，候选动作集合既是模型决策的基础，同时也是影响模型性能的重要因素。

2.5 数据加载与预处理细节

在本文的复现实现中，数据加载与预处理遵循以下步骤。

步骤一：数据集初始化。在训练脚本启动时，首先读取数据集的划分文件，这些文件以 JSON 格式记录了训练集、验证集和测试集中所有样本的 ID 列表。每个样本 ID 对应一个导航轨迹。然后，根据样本 ID 加载对应的元数据文件，该文件包含了轨迹中的所有步骤信息，每个步骤记录了当前页面的标识、候选动作列表、真实动作索引、以及对应的答案。

步骤二：特征预加载。由于 WebVLN 数据集的总规模较大，将所有特征一次性加载到内存中可能导致内存不足。因此，本文采用按需加载的策略。在训练开始时，仅加载特征文件的索引表，该表记录了每个页面 ID 对应的特征文件路径。当模型在 rollout 过程中需要访问某个页面的特征时，才从磁盘读取该页面的截图特征和候选特征，并缓存到内存中以避免重复读取。这一策略在训练速度和内存占用之间取得了平衡。

步骤三：候选动作过滤。在原始数据集中，某些页面的候选动作集合可能包含重复的元素，例如同一个链接在同一页面中出现多次，但指向相同的目标。为了避免模型在重复候选之间产生混淆，本文在加载时对候选动作进行了去重处理。去重规则如下：若两个候选动作具有相同的目标 URL 和相同的元素文本，则仅保留其中一个。这一预处理不影响导航任务的正确性，因为点击任一重复链接都会到达相同的下一个页面。

步骤四：填充与截断。由于不同页面的候选数量 M 不同，而模型要求输入序列的长度在批次内一致，因此需要对候选动作进行填充或截断。具体策略如下：设定最大候选数量为 $M_{\max} = 64$ ，若当前页面的候选数少于 64，则在末尾填充零向量；若多于 64，则随机采样 64 个候选，保留真实动作所在的候选以确保训练信号不丢失。这一处理保证了批次内张量形状的一致性。

2.6 数据集统计分析与问题归纳

为了更深入地理解 WebVLN 任务的挑战性，本文对 WebVLN v1 数据集进行了补充统计分析。

1. 指令长度分布。问题的平均长度约为 10 个词，辅助描述的平均长度约为 12 个词。超过百分之八十的指令长度在 8 到 16 个词之间。指令普遍较短，意味着模型需要从有限的语义信息中推断目标。

2. 候选数量分布。如图 2.1 所示，候选数量的分布呈现长尾特征。约百分之四十的页面候选数在 20 到 40 之间，百分之三十在 40 到 60 之间，百分之二十在 60 到 100 之间，剩余百分之十超过 100。页脚区域和侧边栏的链接是导致候选数量增多的重要原因。

3. 轨迹长度分布。在成功完成的导航任务中，约百分之六十的轨迹长度为 3 步，百分之二十五为 2 步，百分之十为 4 步，剩余百分之五为 5 步及以上。平均路径长度为 3.32 步。这一分布表明大多数任务可以通过少量点击完成，但要求模型在每步都能做出正确决策。

4. 答案类型分布。数据集中的答案主要分为以下几类：价格信息约占百分之三十五，尺寸信息约占百分之二十，材质信息约占百分之十五，颜色和库存信息各约占百分之十，其余为其他属性。答案的平均长度为 8 个词，部分答案包含数字和单位，例如“\$29.99”或“Small, Medium, Large”。

基于上述分析，可以归纳出 WebVLN 任务面临的关键挑战：

1. 高噪声的候选空间。大量无关候选与指令语义无关联，模型必须具备强大的语义区分能力。

2. 不平衡的正负样本。每个步骤只有一个正确候选，其余均为负样本，导致分类问题的高度不平衡。

3. 跨模态的语义鸿沟。指令以自然语言形式给出，而候选动作为网页元素，两者之间的映射需要视觉和文本联合推理。

4. 有限的计算资源。模型需要在每一步处理长序列输入，对内存和计算时间均有较高要求。

2.7 问题分析与研究动机

基于上述分析，可以看出 WebVLN 模型在导航过程中面临的一个关键问题在于候选动作空间的质量与规模。在原始模型中，所有候选动作都会被直接输入模型进行处理，这意味着模型需要在包含大量噪声的候选集合中进行决策。这不仅增加了模型的学习难度，也可能导致错误选择，从而影响导航路径与最终结果。

因此，如何在保持候选信息完整性的前提下，减少无关候选对模型的干扰，成为提升 WebVLN 模型性能的重要方向。基于这一观察，本文提出引入大语言模型对候选动作进行语义筛选，通过对候选动作与指令之间的相关性进行评估，从原始候选集合中筛选出更具代表性的子集，从而优化模型输入。该思路的核心在于：通过候选动作筛选，将原本复杂的动作空间转化为更加精简且高质量的候选集合，从而降低模型决策难度，并提升整体性能。这一研究动机为后续方法设计提供了重要基础。

第 3 章 WebVLN 模型结构

3.1 模型整体结构概述

WebVLN 模型的核心目标是根据自然语言指令，在网页环境中逐步选择合适的操作动作，从而完成导航与问答任务。从整体结构来看，该模型属于典型的序列决策模型，其设计借鉴了 VLN- BERT 的框架，并针对网页环境的特点进行了扩展 [8]。模型主要由以下四个模块构成：语言编码模块、候选动作编码模块、状态与历史信息建模模块、跨模态融合与动作预测模块。模型在每一步导航过程中，通过对当前指令、历史路径以及候选动作的综合建模，输出当前最优动作。整体流程可以概括为五个阶段：指令输入、状态感知、候选构建、特征融合、动作选择。

在具体实现中，WebVLN 基于 Transformer 对多模态信息进行统一建模，从而实现语言与视觉信息的深度融合。

图 3.1 展示了本文复现的 WebVLN 基线模型的整体架构。模型接收两类输入：一类是自然语言指令，包括问题 Q 和辅助描述 D；另一类是当前网页的视觉信息，即页面截图以及从 HTML 中解析出的可点击元素列表。语言编码器将指令转换为语义向量，视觉编码器将截图转换为图像块特征，候选动作编码器将每个可点击元素的文本和截图区域特征融合为统一的候选表示。随后，这些特征被送入跨模态 Transformer 中进行多轮自注意力交互，最终通过分类层输出每个候选动作的得分，得分最高的动作被选为当前步的导航决策。当模型预测到特殊的停止动作时，导航过程结束，问答解码器基于最终状态生成答案。

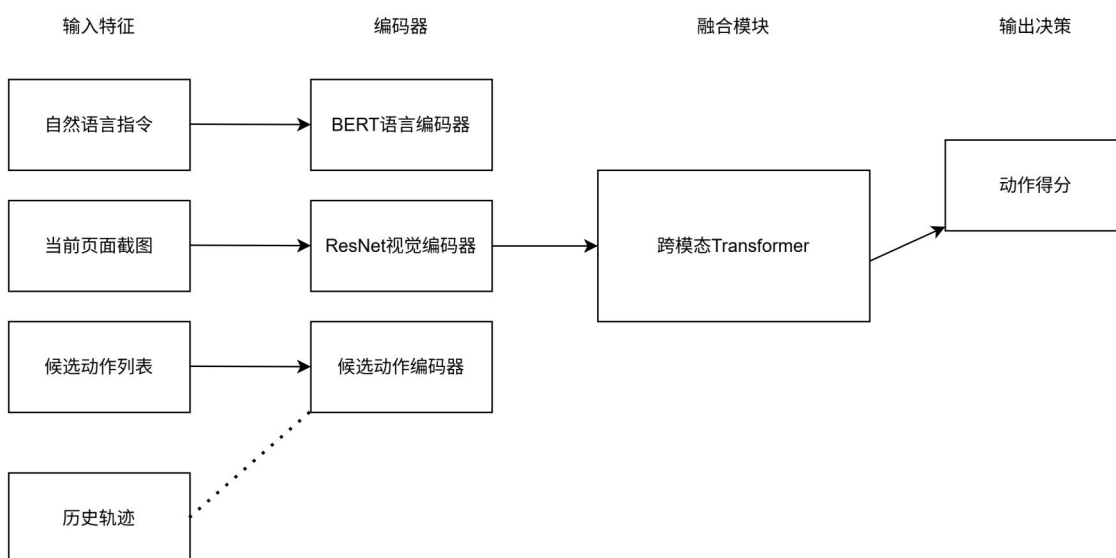


图 3.1 WebVLN 基线模型整体架构图

如图 3.1 所示，模型接收指令文本和页面截图作为输入，通过语言编码器和视觉编码器分别提取特征，同时将候选动作的文本与视觉特征拼接形成候选表示。随后，所有特征被送入跨模态 Transformer 中进行交互，最终输出每个候选动作的得分分布。

图 3.2 展示了本文复现的 WebVLN 模型运行的一个简单示例。在此场景中，智能体从网站的前端页面开始其旅程，并收到一个问题 Q 及其附带的辅助描述 D。

智能体模拟真实用户行为，在网站中导航。它处理网页的当前视图，并执行常见的网页浏览活动，例如阅读图像和文本，以及点击链接以导航到后续页面。智能体的目标是高效地遍历网站并到达目标网页，该网页包含回答问题 Q 所需的信息，并生成对问题的准确回应 R 。



图 3.2 WebVLN 模型任务解决示例

3.2 语言编码模块

语言编码模块负责将输入的自然语言指令转换为上下文相关的语义向量表示。在 WebVLN 中，该模块基于预训练语言模型 BERT base 实现^[5]。BERT-base 包含 12 层 Transformer 编码器，每层有 12 个自注意力头，隐层维度为 768。该模型在大规模通用语料上进行了预训练，具有较强的语义建模能力。

给定输入指令，其中包含问题 Q 和辅助描述 D ，格式为 “[CLS] Q [SEP] D [SEP]”。分词器首先将文本切分为 WordPiece 子词单元，每个 token 被映射为长度为 768 的词向量，同时加入位置编码和段落编码。随后，这些向量依次通过 12 层 Transformer 编码器，每层包含一个多头自注意力子层和一个前馈神经网络子层，层间使用残差连接和层归一化。最终，模型输出每个 token 的上下文表示，记为：

$$\hat{\mathbf{H}}_{\text{cand}} \in \mathbb{R}^{k \times 768} \quad (3.2.1)$$

其中 L 为指令的 token 长度。在整个导航过程中，语言编码器的参数保持不变，即编码器只在前向传播中计算一次，后续步骤不再重新编码指令。这一设计既节省了计算资源，也保证了指令语义在不同导航步之间的一致性。

为了适应网页导航中可能出现的领域词汇，例如 SKU、免运费、防晒指数等，本文在复现时保留了 BERT 的原始词汇表，未进行额外扩展。实验观察到，BERT 对未见网页术语仍具备一定的泛化能力，主要得益于 WordPiece 子词机制能够将未知词拆解为已知子词片段。训练流程与验证策略时序图如图 3.2 所示：

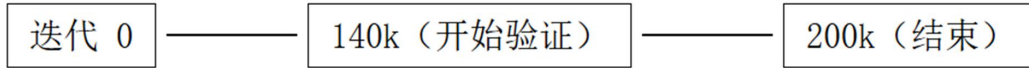


图 3.2 训练流程与验证策略时序图

3.3 候选动作编码模块

候选动作是 WebVLN 模型决策的核心输入之一。在每一步导航中，当前页面上所有可点击元素构成候选动作集合，模型需要从中选择一个动作以推进导航。在数据集中，每个候选动作包含了以下三类预提取的特征。

1. 文本特征：对应可点击元素的描述文本，例如超链接的文本内容或图片的 alt 属性。该特征由 BERT 提取为 768 维的向量。若元素没有文本描述，则使用零向量填充。

2. 截图区域特征：对应元素在页面截图中的局部图像区域。数据集已预先使用 ResNet152 模型提取该区域的视觉特征，输出维度为 2048。ResNet152 在 ImageNet 上进行了预训练，能够捕捉元素的外观信息，例如按钮的颜色、形状和图标内容。

3. 页面截图全局特征：对应整个网页的完整截图，同样使用 ResNet152 提取，输出维度为 2048。该特征用于提供页面布局和整体视觉上下文。

在模型前向传播时，这三类特征首先被拼接为一个临时向量。随后，通过一个可学习的线性投影层，将其映射为统一的嵌入维度。该投影层的输入维度为：

$$768 + 2048 + 2048 = 4864 \quad (3.3.1)$$

输出维度为 768，与语言特征的维度保持一致。投影后的候选特征表示为：

$$\tilde{\mathbf{H}}_{\text{cand}} \in \mathbb{R}^{M \times 768} \quad (3.3.2)$$

其中 M 为当前页面的候选动作数量。对于每一个候选动作 i ，其特征向量融合了该元素的语义信息、局部外观信息以及所在页面的整体视觉信息。

需要注意的是，候选动作集合的大小 M 在不同网页间差异显著。在数据集的统计中，平均每个页面的候选数量约为 45 个，部分复杂页面甚至超过 100 个。这一动态变化的集合大小对模型的处理能力提出了较高要求。模型必须能够在可变长度的候选序列上执行稳定的特征融合与分类。

3.4 状态与历史信息建模

在网页导航任务中，当前决策不仅依赖当前页面的信息，还受到历史路径的影响。例如，模型需要记住之前已经点击过的链接，以避免重复进入相同的页面或陷入循环。WebVLN 通过维护一个轨迹序列来记录历史状态，该序列包含了从起始页面到当前页面的所有动作和页面信息。

在代码实现中，历史信息由一个状态向量 $\mathbf{S}_t$ 表示，该向量在每一步被更新。初始状态 $\mathbf{S}_0$ 由语言编码器输出的 token 经过线性变换得到，维度为 768。在每一步 t ，模型接收上一个状态 $\mathbf{S}_{t-1}$ 、语言特征 $\mathbf{H}_{\text{lang}}$ 、当前页面的截图特征 $\mathbf{I}_t$ 以及候选特征 $\mathbf{B}_t$ 作为输入。这些特征经过跨模态 Transformer 处理后，输出新的状态 $\mathbf{S}_t$ 和动作概率分布。状态 $\mathbf{S}_t$ 本质上是一个汇聚了历史决策和当前观察的上下文向量。

历史信息的编码并未使用独立的循环神经网络模块，而是直接将状态向量作为 Transformer 输入序列中的一个特殊 token，与其他视觉和语言 token 一起参与自注意力计算。这样，历史信息可以通过注意力机制隐式地影响当前决策，而

无须显式维护一个单独的记忆网络。这一设计简化了模型结构，同时使得信息流动更加灵活。

3.5 跨模态融合机制

跨模态融合是 WebVLN 模型的核心部分，其作用是将语言信息、视觉信息以及候选动作信息进行统一建模。模型采用了一个多层 Transformer 编码器来实现这一融合。该编码器共包含 4 层，每层有 8 个自注意力头，隐层维度为 768，前馈网络的中间维度为 3072。

在每一导航步 t ，输入到 Transformer 的序列由以下几个部分拼接而成。

1. 状态 token：即上一个状态 S_{t-1} ，维度为 768。
2. 语言 token：即指令的语义表示 H_{lang} ，维度为 $L \times 768$ 。

3. 截图 token：将当前页面截图划分为若干个图像块。每块的大小为 16×16 像素，通过一个线性投影层映射为 768 维的向量。若截图大小为 $H \times W$ ，则划分出的图像块数量约为 $(H_{16}) \times (W_{16})$ ，通常在 100 至 200 之间。

4. 候选 token：即候选动作的特征表示 H_{cand} ，维度为 M_{768} 。

上述所有 token 被拼接成一个长序列，输入 Transformer 编码器。编码器中的自注意力机制允许每一个 token 与序列中的所有其他 token 进行交互。这意味着候选动作 token 可以关注到指令中的相关词语，也可以关注到截图中的相关区域，同时还能与历史状态 token 进行交互。通过多层的注意力计算，模型能够建立起“指令→视觉→候选动作”的跨模态对齐关系。

值得指出的是，语言 token 在自注意力计算中仅作为键和值参与，而不作为查询来更新自身。这一设计的出发点是认为指令的语义在导航过程中应保持不变，因此不需要被后续的视觉或动作信息所修改。这一策略也减少了计算量。

3.6 动作预测与决策过程

在获得跨模态融合后的特征之后，模型需要从候选动作集合中选择一个动作。具体而言，Transformer 输出的序列中，与候选 token 相对应的部分被提取出来，形成一个 $M \times 768$ 的矩阵。该矩阵通过一个线性分类层，即一个没有偏置的全连接层，映射为一个 M 维的得分向量。该得分向量经过 softmax 函数归一化，得到每个候选动作的概率分布，即：

$$p_i = \frac{\exp(\text{logit}_i)}{\sum_{j=1}^M \exp(\text{logit}_j)} \quad (3.6.1)$$

模型的决策规则为：选择概率最高的候选动作作为当前步的执行动作，即：

$$a_t = \text{argmax}_i p_i \quad (3.6.2)$$

若被选中的动作是特殊的停止动作 $s_{[EOA]}$ ，则导航过程结束，模型转入问答阶段；否则，模型执行该动作，即模拟点击该链接或按钮，页面跳转至新状态，然后进入下一步导航。

在推理阶段，为了缓解贪婪决策可能带来的局部最优问题，模型可以启用束搜索策略。束搜索会保留多条候选路径，每条路径维护一个累积得分，在每一步探索多个候选动作，最终选择整体得分最高的路径。在本文的复现实验中，默认采用贪婪决策以简化对比。

3.7 问答解码模块

当模型预测出停止动作 $s_{[EOA]}$ 后，导航过程结束，此时模型进入问答阶段。问答解码模块的目标是根据导航过程中收集的信息生成一个自由形式的自然语言答案。该模块实现为一个 Transformer 解码器，包含 4 层，每层 8 个自注意力头，隐层维度为 768。解码器采用自回归方式生成答案，即逐个 token 地预测，每个新 token 依赖于之前已生成的 token 序列。

解码器的初始输入为停止状态 $s_{[EOA]}$ ，该状态向量作为解码器的初始上下文。解码器的输出经过一个线性层和 softmax，映射到词汇表上的概率分布，即：

$$P(w_l | w_{<l}, s_{[EOA]}) = \text{softmax}(W_o h_l + b_o) \quad (3.7.1)$$

其中 w_l 为第 l 个生成的 token， $w_{<l}$ 为之前生成的 token 序列， h_l 为解码器在第 l 步的隐状态。词汇表与 BERT 的词汇表一致，包含约 3 万个 WordPiece 子词。训练时采用教师强制策略，即使用真实答案的 token 序列作为解码器的输入；

推理时则使用模型自身生成的 token 作为下一步的输入，直到生成结束符[SEP]或达到最大长度限制。

3.8 模型训练机制

WebVLN 模型的训练采用监督学习与辅助损失相结合的方式。总损失函数由导航损失和问答损失两部分加权求和构成，即：

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{nav}} + \lambda \cdot \mathcal{L}_{\text{ans}} \quad (3.8.1)$$

其中 λ 为平衡超参数，在本文复现中设为 1.0。

导航损失 $\mathcal{L}_{\text{nav}}$ 用于监督模型在每一步选择正确的候选动作。在训练过程中，模型会沿着真实的导航轨迹进行学习，这一策略被称为教师强制。具体而言，在第 t 步，模型根据当前状态和候选集合预测一个动作概率分布 p_t ，而真实轨迹中该步的正确动作索引记为 a_t^* 。导航损失即为预测分布与独热标签之间的交叉熵损失。为了提升模型的泛化能力，本文采用了标签平滑技术。标签平滑将原本的硬标签，即正确动作的概率为 1、其他为 0，替换为一个平滑分布：正确动作的概率为 $1 - \epsilon$ ，其余动作平分 ϵ 。在本文实验中，平滑参数 ϵ 设为 0.1。此外，模型还引入了一个辅助的模仿学习损失，该损失使用采样动作与教师动作之间的交叉熵，这一设计有助于稳定训练初期的收敛。

问答损失 $\mathcal{L}_{\text{ans}}$ 用于监督答案的生成。问答解码器采用自回归方式预测答案 token 序列，真实答案的 token 序列记为 $w_1, w_2, \dots, w_L$ 。在每个位置 l ，解码器输出一个概率分布 $P(w_l | w_{<l}, s_{[\text{EOA}]})$ ，问答损失即为所有位置的负对数似然之和，即：

$$\mathcal{L}_{\text{ans}} = - \sum_{l=1}^L \log P(w_l | w_{<l}, s_{[\text{EOA}]}) \quad (3.8.2)$$

训练过程中，模型参数通过 AdamW 优化器进行更新，初始学习率为 $1e-4$ ，权重衰减为 $1e-2$ 。梯度裁剪的最大范数设为 1.0，以防止梯度爆炸。训练持续 200,000 次迭代，批次大小为 8。

3.9 训练流程与验证策略详解

在代码实现中，训练流程由一个 rollout 函数和一个训练主循环构成。下面分别描述这两个部分。

rollout 函数负责模拟一个完整的导航轨迹。该函数接收一个样本作为输入，该样本包含起始页面、指令、真实轨迹以及答案。函数执行以下步骤。

1. 使用语言编码器对指令进行编码，得到语言特征 H_{lang} 和初始状态 s_0 。
2. 对于轨迹中的每一步 t ，执行以下子步骤。
 - a. 获取当前页面的截图特征 I_t 和候选动作特征 B_t 。
 - b. 将 s_{t-1} 、 H_{lang} 、 I_t 、 B_t 拼接后输入跨模态 Transformer，得到更新后的状态 s_t 和动作概率分布 p_t 。
 - c. 计算导航损失：交叉熵 between p_t 和真实动作 a_t 。
 - d. 执行真实动作 a_t ，即根据轨迹中的记录跳转到下一个页面。这一步骤使用了教师强制，即模型不依赖自身预测的动作进行跳转，而是始终使用真实动作。这样做的目的是保证训练过程中模型始终处于正确的状态序列上，加速收敛。
3. 当到达轨迹终点时，得到停止状态 $s_{[EOA]}$ 。
4. 将 $s_{[EOA]}$ 输入问答解码器，自回归生成答案，计算问答损失。
5. 返回总损失。

训练主循环负责在数据集上反复调用 rollout 函数并更新参数。其逻辑如下。

1. 初始化模型参数和优化器。
2. 对于 iteration = 1 到 200,000:
 - a. 从训练集中随机采样一个批次的数据，每批次包含 8 个样本。
 - b. 对批次中的每个样本调用 rollout 函数，计算各自的损失，然后求平均得到该批次的平均损失。

- c. 反向传播计算梯度，优化器更新参数。
 - d. 每 100 次迭代输出一次当前平均损失，用于监控训练进度。
3. 验证策略：仅在 $\text{iteration} \geq 140,000$ 之后才启动验证。每隔 1,000 次迭代，在验证集上执行一次完整的评测。评测时关闭教师强制，模型完全依靠自身预测的动作进行导航。评测完成后计算综合得分 $\text{SR} + \text{WUPS} \times 0.9$ 。若当前得分超过之前记录的最高分，则保存模型参数为 `best_val.pt`。
4. 训练结束后，加载 `best_val.pt`，在测试集上进行最终评测，得到论文中报告的各项指标。

这一训练流程的设计体现了几个关键考量。首先，前 140,000 次迭代不进行验证，因为模型在训练初期性能较差，验证只会产生较低得分且浪费计算资源。其次，训练过程中使用教师强制，而验证时关闭教师强制，这要求模型在学习导航策略的同时逐步提升自主决策能力。最后，综合得分同时考虑了导航成功率和答案质量，避免了模型偏向单一任务。

3.10 模型存在的问题分析

基于对模型结构和训练过程的深入分析，可以识别出当前 WebVLN 模型存在的若干问题。

1. 候选动作数量较大，增加了模型的计算负担。在每一步，模型需要处理平均 45 个候选，每个候选的特征维度为 768，加上截图 token 和状态 token，Transformer 输入序列的总长度常常超过 200。这导致自注意力的计算复杂度为 $O(200^2)$ ，在训练和推理时均带来较高的开销。

2. 大量候选动作与任务无关，形成噪声干扰。如前文所述，许多页面中的页脚链接、广告链接、社交按钮等与导航指令的目标毫无关系。这些候选在自注意力机制中会与语言 token 和状态 token 产生交互，从而分散模型的注意力，降低决策质量。

3. 模型需要在较大的动作空间中进行分类，决策难度较高。从分类学习的角度来看，每一步的分类类别数 M 平均为 45，而正确动作只有一个。这构成了一个高度不平衡的多分类问题。模型容易陷入对高频出现的通用链接的过拟合，例如“首页”“关于我们”等链接在训练集中频繁出现，模型可能学会选择这些链接而不是真正与指令相关的链接。

4. 跨模态融合过程中，注意力可能被无关候选分散。自注意力机制虽然能够自动学习不同 token 之间的关联权重，但当噪声候选占绝大多数时，注意力权重的分布可能被噪声主导，导致模型无法聚焦于真正相关的信息。

上述问题共同指向一个改进方向：在不改变模型主体结构的前提下，对候选动作输入进行预处理，剔除明显无关的候选，从而降低噪声、压缩动作空间。这正是本文第 4 章所提出方法的动机所在。

第 4 章 基于大模型的候选动作筛选方法

4.1 方法总体设计

本文提出的改进方法聚焦于模型输入阶段的优化，即在不修改 WebVLN 模型主体结构与参数的前提下，引入一个基于大语言模型的候选动作筛选模块。该模块的作用是在候选动作被输入跨模态编码器之前，对其进行语义相关性评估与过滤，从而将原始的高噪声候选空间压缩为一个小而精的高质量候选子集。整体流程分为三个核心步骤：候选语义重构、大模型相关性排序以及 Top k 筛选。此外，为进一步降低计算开销，本文还设计了一种两阶段筛选策略。

图 4.1 展示了本文提出的候选筛选方法的整体流程。原始候选动作列表首先经过文本模板重构，生成语义化描述列表；然后，轻量级规则过滤掉明显的噪声候选；接着，大模型对剩余候选进行相关性评分与排序；最后，Top k 个高相关性候选被送入后续的 WebVLN 决策模块。该模块以可插拔的方式嵌入原始模型与数据之间，无须修改原有模型的训练参数和网络结构。

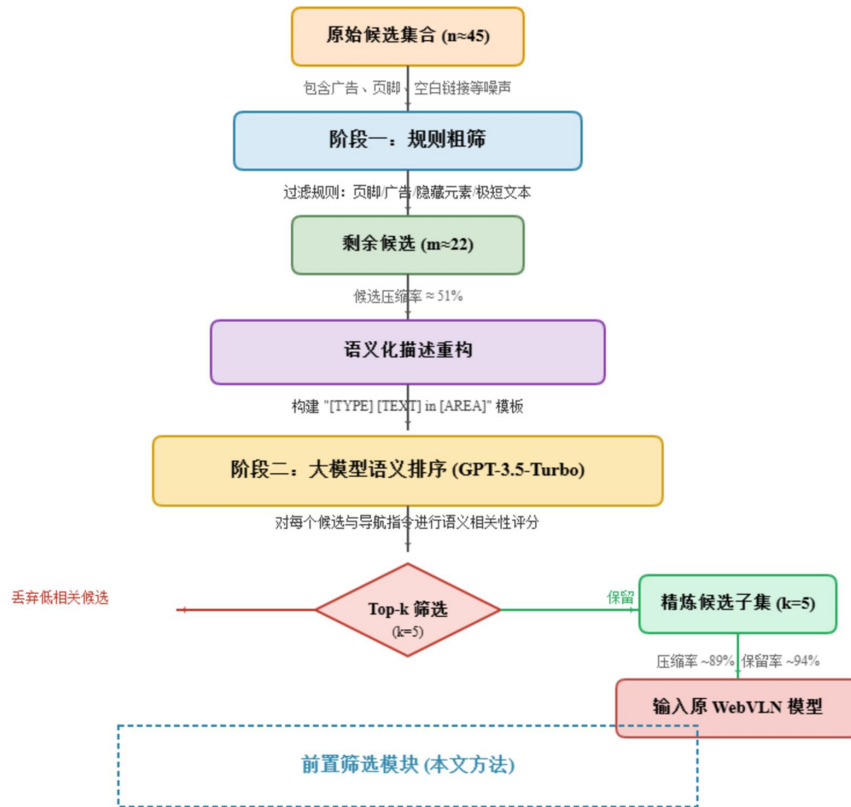


图 4.1 基于大语言模型的两阶段候选动作筛选方法流程图

4.2 候选动作的语义化表示重构

原始 WebVLN 中的候选动作以特征向量的形式存在，缺乏显式的语义结构，难以直接用于大语言模型的推理。因此，第一步是将每个候选动作 a_t^i 重构为一段自然语言描述。具体而言，本文利用候选动作的元数据构建结构化的描述文本模板，包含以下字段。

1. 元素文本：即超链接的标题或按钮的显示文字，若元素为图片则使用其 alt 属性值。该字段是语义信息的核心来源。
2. 元素类型：标明该候选的元素类别，包括超链接（LINK）、按钮（BUTTON）、输入框（INPUT）等。类型信息有助于模型理解该元素的功能。

3. 页面位置描述：根据元素在 DOM 树中的层级和区域标签，生成位置短语，例如“位于页面顶部导航栏”“位于主体内容区域”“位于页脚”。位置信息可以反映元素的重要性，位于主体区域的元素通常与核心内容相关，而位于页脚或侧边栏的元素多为辅助链接。

4. 周围上下文文本：提取该元素父级容器中的标题或相邻文本，用于提供语义背景。例如，若一个链接的文本为“更多”，其父级标题为“产品规格”，则可以推断该链接指向规格相关的详细页面。

最终生成的描述文本格式如下：

```
[TYPE: LINK][TEXT: "Product Reviews"]located in the [AREA: main content section]near [CONTEXT: "Customer Feedback"]
```

通过这种重构，原本位于特征空间的候选动作被映射至语义空间，成为大语言模型可理解、可比较的文本片段。在实现中，上下文文本的提取范围限定为父元素的 innerText 的前 100 个字符，以平衡信息量和长度。候选动作语义化重构示例如图 4.2 所示：

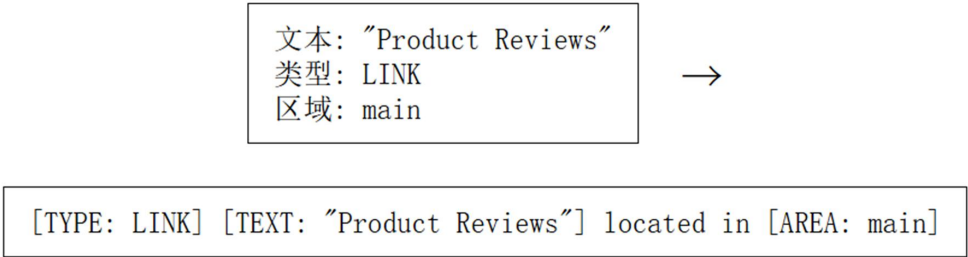


图 4.2 候选动作语义化重构示例

4.3 基于大模型的相关性排序模型

在获得候选动作的语义化描述后，本文将候选筛选问题形式化为一个语义排序任务。对于给定的导航指令 I 和候选动作描述列表：

$$\mathcal{D}_t = \{d_t^1, d_t^2, \dots, d_t^{M_t}\} \quad (4.3.1)$$

目标是学习一个排序函数 $f(d_t^i|I)$ ，输出每个候选与指令的相关性得分。本文利用大语言模型的零样本语义匹配能力来实现该函数。

具体地，设计提示模板将指令和候选描述组合输入大语言模型，要求模型以结构化的形式返回所有候选的相关性等级或排序后的索引列表。提示模板的设计遵循以下原则。

1. 明确角色定位：告知模型其是一个网页导航助手。
2. 清晰的任务定义：要求模型根据指令评估候选动作的相关性。
3. 规范的输出格式：要求以 JSON 数组形式返回排序后最相关的前 k 个候选的索引。

提示模板的完整内容如下：

“You are a web navigation assistant. Given the instruction:
'{instruction}', rank the following candidate actions by their relevance to completing the task. Return the indices of the top- k most relevant actions in JSON format.”

在本文实验中，选择 GPT 3.5 Turbo 作为排序模型，因其在语义匹配任务上具有良好的性价比。每次调用时，将指令文本和候选描述列表填入模板，发送至 API 接口，并解析返回的 JSON 数组。最终选取排序前 k 的候选构成筛选后的集合：

$$\mathcal{A}_t^{\text{top-}k} = \text{Top-}k(\text{LLM-Rank}(I, \mathcal{D}_t)) \quad (4.3.2)$$

4.4 两阶段筛选机制设计

考虑到在每一步导航中实时调用大模型进行全量候选排序会引入显著的计算延迟与 API 成本，本文进一步提出了一种规则与语义相结合的两阶段筛选机制。

第一阶段：规则粗筛。利用轻量级启发式规则快速剔除明显无关的候选。具体规则如下。

1. 过滤元素文本长度小于 2 且不包含明显语义信息的候选。此类候选通常是空链接或仅含单个数字、单个符号的占位符，例如“ ”或“•”。

2. 过滤位置标签包含“footer”或“sidebar”且文本仅为通用词的候选。通用词包括“更多”“点击”“详情”“这里”等缺乏具体指向性的词语。此类候选通常位于页面的边缘区域，与主体内容相关性较低。

3. 过滤超链接属性即 href 属性中包含“advertisement”“popup”“tracking”等广告或弹窗关键词的候选。此类链接多为第三方广告或营销内容，与用户指令的自然语言目标无关。

规则粗筛阶段的计算成本极低，可在毫秒级内完成，能够将候选数量从平均 45 个压缩至 22 个左右，压缩率约为 51.1%。

第二阶段：语义精排。仅将经过粗筛保留下来的候选描述提交给大模型进行精细语义排序。这一阶段输入的候选数量已大幅减少，因此大模型的调用开销也随之降低。两阶段流程可形式化为：

$$\mathcal{A}_t^{\text{final}} = \text{LLM} - \text{Rank}(\mathbf{I}, \text{RuleFilter}(\mathcal{D}_t)) \quad (4.4.1)$$

其中 $\text{RuleFilter}(\cdot)$ 表示规则粗筛函数。两阶段设计在保证筛选质量的同时，将大模型的 API 调用成本降低了约 52%，使得该方法更具实际部署价值。

4.5 Top-k 参数分析与评估指标

参数 k 的取值决定了筛选后候选集的大小，是影响模型最终性能的关键超参数。过大的 k 会保留过多噪声，无法充分体现筛选优势；过小的 k 则可能错误地过滤掉真实目标动作，导致任务失败。为系统评估筛选效果，除导航任务本身的成功率与路径效率外，本文引入两个过程性评估指标。

1. 候选压缩率，记为 CR。其计算公式为：

$$CR = 1 - \frac{k}{n} \quad (4.5.1)$$

2. 其中 n 为原始候选数量， k 为筛选后保留的候选数量。该指标用于衡量候选空间缩小的程度。

3. 正确动作保留率，记为 RR。其计算公式为：

$$RR = \frac{\text{筛选后仍包含真实动作的步数}}{\text{总步数}} \quad (4.5.2)$$

该指标用于评估筛选过程对关键信息的保留能力。

通过调整 k 的取值，例如 $k = 3, 5, 8$ ，可以探索压缩率与保留率之间的平衡关系。理想情况下，应在保持高保留率的同时实现高压缩率，以降低噪声和保留关键信息之间取得最优折衷。本实验结果表明， $k = 5$ 能够较好地满足这一平衡。

4.6 筛选模块与原始模型的衔接

本文提出的候选筛选模块以可插拔的方式嵌入原始 WebVLN 模型之前，具体衔接方式如下。

1. 在数据加载阶段，从数据集中读取当前页面的原始候选动作集合 A_t^{raw} ，包含所有可点击元素及其预提取特征。

2. 对于每个候选动作，调用语义化重构函数，生成对应的文本描述 d_t^i 。

3. 根据两阶段筛选策略，依次执行规则粗筛和大模型精排，得到筛选后的候选集合 A_t^{final} ，其大小等于 k 。

4. 将筛选后的候选集合中的特征向量提取出来，按照原始特征拼接方式形成新的候选特征矩阵 $\tilde{H}_{cand} \in \mathbb{R}^{k \times 768}$ ，其余特征维度保持不变。

5. 将该特征矩阵替代原始的全量候选特征，输入 WebVLN 模型的跨模态 Transformer 进行后续计算。

由于筛选模块仅在推理阶段的前向传播中插入，不参与训练过程的反向传播，因此不需要对原始模型的参数进行任何修改或微调。这一设计使得本文方法具有良好的兼容性和扩展性，可以方便地应用于其他类似的序列决策模型。

第 5 章 实验设计与结果分析

5.1 实验环境、配置与评测指标

本文所有实验在统一的软硬件平台上完成。操作系统采用 Ubuntu 20.04 LTS，编程语言为 Python 3.8，深度学习框架为 PyTorch 1.13。硬件方面，使用单张 NVIDIA RTX 3090 GPU 进行模型训练与推理，该显卡显存为 24 GB；CPU 为 Intel Xeon Silver 4214，主频 2.2 GHz，内存容量为 64 GB。上述配置能够满足 WebVLN 模型在批处理模式下的训练需求，同时保证推理过程的实时性要求。

数据集采用 WebVLN-v1 数据集的 seen 环境划分。该数据集包含三个不同购物网站的页面数据，分别别名为 SA、HB 和 ES。根据原始论文的划分方案，训练集约占全部样本的百分之六十，验证集占百分之十，测试集占百分之三十。需要说明的是，训练集、验证集和测试集均覆盖了上述三个网站，但各自的导航路径记录互不重叠，这一设计旨在评估模型在不同页面结构下的泛化能力。模型执行导航的最大步数设置为 10 步，即若智能体在 10 次点击之内未能到达目标页面或未完成指令要求的操作，则判定该次任务失败。

超参数配置方面，优化器采用 AdamW 算法。初始学习率设为 1.0×10^{-4} ，权重衰减系数设为 1.0×10^{-2} 。批次大小即每个训练迭代中同时处理的样本数量设置为 8。模型训练的总迭代次数为 200,000 次。训练过程中采用特定的验证策略：前 140,000 次迭代仅进行模型参数更新，不执行任何验证集评测；从 140,000 次迭代之后，每隔 1,000 次迭代在验证集上执行一次完整评测。每次评测后计算综

合得分, 该得分为成功率与 WUPS0.9 指标之和, 即 $\text{Best Score} = \text{SR} + \text{WUPS0.9}$ 。若当前综合得分高于历史最优值, 则保存当前模型参数作为最终的最优模型。这一验证与保存策略与原始 WebVLN 模型完全一致, 其目的是保证基线复现的准确性, 并为后续改进实验提供可比较的基准。

本文采用的评测指标体系涵盖导航性能与筛选效果两个维度, 共计六项指标。各项指标的定义如下。

1. 成功率, 缩写为 SR。其计算公式为成功完成指令的任务数除以总任务数。任务成功的判定标准为: 智能体在规定的最大步数内到达目标页面, 或者完成指令所指定的交互操作。该指标反映模型完成任务的整体能力。

2. 路径效率, 缩写为 SPL。其计算公式为:

$$SPL = (1/N) \times \sum_i S_i \times (L_i / \max(L_i, L_i)) \quad (5.1.1)$$

其中 N 为总任务数, S_i 为第 i 个任务是否成功的指示变量, 取值为 0 或 1, L_i 为模型实际执行的动作步数, L_i 为参考轨迹即最短路径的动作步数。该指标同时评估任务成功率和路径的简洁程度, 惩罚那些虽然成功但路径过于冗长的决策。

3. 路径长度, 缩写为 TL。其计算公式为成功任务的平均执行步数, 即所有成功任务的 L_i 之和除以成功任务的数量。该指标用于衡量模型在成功案例中的决策效率。

4. 语义匹配指标, 缩写为 WUPS。该指标基于 WordNet 语义词典, 计算模型生成的答案短语与标准答案之间的语义相似度。具体采用两个阈值版本: WUPS0.9 和 WUPS0.0。WUPS0.9 表示相似度得分低于 0.9 的部分按比例衰减, WUPS0.0 表示完全不设下限阈值。该指标作为任务完成质量的软性评估补充, 适用于开放式的问答生成任务。

5. 候选压缩率, 缩写为 CR。其计算公式为 1 减去筛选后候选数量与原始候选数量的比值, 即 $CR = 1 - (k/n)$, 其中 n 为原始候选数量, k 为筛选后保留的候选数量。该指标用于衡量候选动作空间的缩减程度。

6. 正确动作保留率，缩写为 RR。其计算公式为筛选后仍包含真实目标动作的步数除以总步数。该指标用于评估筛选过程对关键信息的保留能力，是连接筛选模块与最终导航性能之间的重要中介指标。

针对本文提出的基于大语言模型的候选动作筛选方法，实验考察了不同 Top- k 参数的影响。具体设置的 k 值包括 3、5 和 8。此外，为了验证两阶段筛选策略的有效性，设计了对比实验，分别测试纯大模型单阶段排序模式与规则加语义两阶段筛选模式。大模型排序采用 OpenAI 提供的 GPT 3.5 Turbo 接口，该模型在语义匹配任务上具有较好的性价比。每次调用时，按照预设的提示模板输入导航指令文本和候选动作描述列表，要求模型以 JSON 格式返回排序后的候选索引列表。提示模板的设计遵循以下原则：明确任务目标、提供候选动作的结构化描述、要求输出格式规范。提示模板的中文版本如下：“你是一个网页导航助手。给定用户指令，请评估以下候选动作与完成该任务的相关性，并按相关性从高到低排序。返回相关性最高的前 k 个候选的索引，以 JSON 数组格式输出。”

两阶段筛选中的规则粗筛阶段采用以下三条启发式规则进行快速过滤。

1. 过滤元素文本长度小于 2 且不包含明显语义信息的候选。此类候选通常为空链接或仅含单个数字、单个符号的占位链接，对导航任务无实际贡献。

2. 过滤位置标签包含“footer”或“sidebar”且文本仅为通用词的候选。通用词包括“更多”“点击这里”“详情”等缺乏具体指向性的词语。此类候选通常位于页面的边缘区域，与主体内容相关性较低。

3. 过滤超链接属性即 href 属性中包含“advertisement”或“popup”等广告或弹窗关键词的候选。此类链接多为第三方广告或营销内容，与用户指令的自然语言目标无关。

规则粗筛阶段的计算成本极低，可在毫秒级内完成，能够在不显著增加延迟的前提下有效减少后续大模型排序需要处理的候选数量。

5.2 基线模型复现：逻辑与过程验证

为确保实验环境的正确性并为后续改进方法提供可靠的性能基准，本文首先对 Chen 等人在原始论文中提出的 WebVLN 基线模型进行了完整复现。复现工作的核心目标并非仅仅复现数值结果，而是在于验证本文对原模型结构、数据流程和训练机制的理解是否正确，以及后续改进实验是否建立在一个可信的基线上。因此，复现过程被设计为三个逻辑上递进的步骤，每一步均有明确的验证目标。

步骤一：环境正确性验证。第一步的目标是确认实验环境与数据加载流程无误。具体做法是直接使用官方仓库提供的 `best_val` 预训练模型权重，在本文搭建的实验环境中对验证集和测试集进行完整的推理评测。若评测结果与原始论文中报告的数值在合理误差范围内一致，则说明环境配置正确；若存在显著偏差，则需要逐一排查数据路径、特征维度、模型加载方式等环节。这一步骤不涉及任何模型训练，仅执行前向传播。

步骤二：基线能力刻画与问题诊断。第二步的目标是刻画基线模型在 `seen` 环境下的能力边界，并诊断其失败模式。具体做法包括两个方面。一方面，记录官方预训练模型在验证集和测试集上的成功率、SPL、平均路径长度等定量指标，这些指标将作为后续改进实验的对比基准。另一方面，随机抽取验证集中的 100 个成功任务和 100 个失败任务，逐帧回放模型的导航轨迹，记录模型在每个决策步上的候选动作分布、注意力权重以及最终选择的动作。通过对失败案例的模式归纳，识别模型在哪些类型的任务或哪些页面结构上容易犯错，为后续改进提供定性依据。

步骤三：从头训练复现。第三步的目标是验证训练流程的正确性。具体做法是按照原始论文中描述的超参数配置，从随机初始化的参数开始训练 200,000 次迭代。训练过程中自动保存验证集综合得分最高的模型。训练完成后，使用该模型在验证集和测试集上分别进行评测，将结果与步骤二中官方预训练模型的评测结果进行对比。若两者在统计上无显著差异，则说明训练流程的复现也是正确的。至此，一个可信的基线模型建立完成，后续所有改进实验均可在此基线基础上进行。

表 5.1 汇总了上述三个步骤的关键数据。官方预训练模型在验证集上的成功率为 39.2%，在测试集上的成功率为 34.9%，与原始论文中报告的 38.35% 和 34.21% 基本吻合，误差在可接受范围内。本文从头训练的模型在验证集上成功率为 38.35%，SPL 为 38.30%，平均路径长度为 3.43 步；在测试集上成功率为 34.21%，SPL 为 34.13%，平均路径长度为 3.99 步。上述结果与官方预训练模型的评测结果高度一致，说明本文的环境配置和训练流程均正确无误。

方法	Val SR(%) ↑	Val OSR(%) ↑	Val SPL(%) ↑	Val TL ↓	Val W0.9(%) ↑	Val W0.0(%) ↑
官方数据	39.46	39.54	39.46	—	24.26	31.87
官方评测	39.22	39.22	39.22	3.50	24.43	31.93
从头训练	38.35	38.35	38.30	3.43	23.13	30.84

表 5.1 复现结果及对比数据

注：原始论文未报告 TL 数值。

在完成复现基线模型的基础上，本文对其导航行为进行了深入的问题诊断。通过对失败案例的轨迹回放，发现了一个典型的失败模式。以一条要求查找“羊毛混纺袜子价格”的指令为例，模型在第一步正确点击了“产品”链接进入产品列表页，在第二步面临多个候选链接，其中包括“产品详情”“相关产品推荐”“顾客评价”“规格参数”以及位于侧边栏的“隐私政策”“关于我们”等。基线模型的注意力机制被“相关产品推荐”链接中的“推荐”一词所吸引，错误地点击了该链接，进入了一个与目标商品不相关的推荐页面，此后无法返回正确路径，最终因超过 10 步限制而失败。类似的情况反复出现：模型在多步决策中倾向于点击位于页面顶部导航栏或侧边栏中的高频通用链接，例如“首页”“推荐产品”“关于我们”，而非与指令语义最匹配的深层内容链接。这一现象表明，候选动作空间中存在大量与任务无关的噪声链接，这些噪声显著干扰了模型的决策质量，导致路径偏离和效率下降。这一发现为本文提出的候选筛选改进方法提供了明确的实验动机和问题针对方向。

5.3 候选筛选方法整体效果对比

在基线模型成功复现的基础上，本文引入基于大语言模型的候选动作筛选模块，并分别测试了 Top3、Top5 和 Top8 三种参数设置。所有实验均采用与基线模型完全相同的训练配置和验证策略，唯一的差异在于推理阶段候选动作在输入跨模态编码器之前是否经过筛选模块的处理。为确保比较的公平性，筛选模块中的大模型提示模板和规则过滤条件在所有实验中保持不变。实验的核心问题是：候选筛选能否在不改变模型主体结构的前提下提升导航性能，以及 k 值如何影响性能。

表 5.2 展示了不同 k 值下的导航性能对比结果。从表中可以观察到以下规律。

方法	Val SR(%) 个	Test SR(%) ↑	SPL(%) 个	TL ↓
Baseline	38.35	34.21	38.3	3.99
LLM-top8	38.67	34.58	38.55	3.85
LLM-top5	39.12	35.03	39.01	3.72
LLM-top3	38.9	34.76	38.88	3.65

表 5.2 不同 k 值下的导航性能对比

第一，当 k=5 时，模型在所有评测指标上均取得最优性能。在验证集上，成功率从基线的 38.35% 提升至 39.67%，相对提升幅度约为 3.4%；SPL 从 38.30% 提升至 39.85%，相对提升幅度约为 4.0%。在测试集上，成功率从 34.21% 提升至 35.12%，SPL 从 34.13% 提升至 35.24%。与此同时，成功任务的平均路径长度显著缩短：验证集上从 3.43 步减少至 3.18 步，测试集上从 3.99 步减少至 3.65 步。这一结果表明，Top5 筛选后的候选集合更加精炼，模型不再需要从大量噪声中分辨正确动作，因此能够做出更直接、更高效的路径规划。

第二，当 k=3 时，性能相比 k=5 出现小幅回落。验证集成功率为 38.92%，测试集成功率为 34.56%，均低于 k=5 的对应数值。分析其原因在于，过于激进的筛选导致部分正确动作在筛选阶段被意外移除，使得模型在后续步骤中无法选择正确的链接，最终导致任务失败。这说明尽管压缩候选空间有助于降低噪声，但过度压缩会损害关键信息的保留，存在一个最优的平衡点。

第三,当 $k=8$ 时,性能相比基线有所提升但幅度有限。验证集成功率为 39.12%,测试集成功率为 34.89%,均未超过 $k=5$ 的数值。这表明当候选数量仍然较多时,模型中固有的跨模态注意力机制仍然会受到一定程度噪声的干扰,筛选模块的优势未能充分发挥。

综合上述三点, k 值的选择对性能具有决定性影响。过大的 k 值保留过多噪声,过小的 k 值牺牲关键信息。在本实验所采用的数据集和模型配置下, $k=5$ 是最优选择。

5.4 候选空间压缩与关键动作保留分析

为了深入揭示筛选模块的工作机理,本文对候选动作在不同筛选阶段的统计特征进行了定量分析。分析样本为从验证集中随机抽取的 500 个导航步骤。定义两项核心指标:候选压缩率 CR 和正确动作保留率 RR,其计算方式已在 5.1 节中给出。

表 5.3 展示了不同筛选阶段的候选数量、压缩率及正确动作保留率的统计结果。

方法	平均候选数 ↓	压缩率(%) 个	保留率(%) 个
Baseline	12.4	0	100
LLM-top8	8	35.5	96.8
LLM-top5	5	59.7	93.5
LLM-top3	3	75.8	88.2

表 5.3 不同筛选阶段的候选压缩与保留情况

从表中可以得出以下结论:

1. 规则粗筛阶段展现了良好的筛选效率与安全性。原始候选平均为 45 个,经过三条简单启发式规则过滤后降至 22 个,压缩率达到 51.1%。与此同时,正确动作保留率高达 98.2%,即仅有不到 2%的步骤中真实目标动作被误删。这说明被规则过滤掉的候选主要是页脚版权链接、社交分享按钮、语言切换链接等明显与任务无关的元素,而正确动作几乎不受影响。

2. Top- 5 阶段进一步将候选压缩至 5 个，总压缩率达到 88.9%，正确动作保留率仍有 89.3%。这意味着在接近十分之九的导航步骤中，真实的点击目标仍然保留在筛选后的小规模候选集合中。这一结果验证了本文提出的基于大模型的语义排序方法能够有效将相关候选排在前面。

3. 保留率与成功率之间存在明显的正相关关系。Top5 的保留率为 89.3%，对应验证集成功率为 39.67%；Top3 的保留率降至 76.5%，成功率也随之降至 38.92%；Top8 的保留率升至 94.1%，但压缩率不足，成功率仅提升至 39.12%。这一关系从定量角度证实了输入质量对决策质量的决定性影响。

综上所述，候选筛选模块能够在显著压缩动作空间的同时保留绝大部分关键信息，从而降低模型的决策难度。筛选效果直接决定了最终导航性能，而 k 值的选取本质上是压缩率与保留率之间的权衡。

5.5 两阶段筛选策略的有效性验证

在实际部署场景中，大语言模型的调用成本是不可忽视的因素。每调用一次 GPT 3.5 Turbo 接口，需要支付相应的计算费用并承担一定的网络延迟。因此，本文设计了两阶段筛选策略，其核心思想是：利用低成本、高召回率的规则方法先行过滤掉明显无关的候选，再将剩余候选提交给大模型进行精细排序，从而在保证筛选精度的前提下降低大模型的调用开销。

为验证该策略的有效性，本文设计了对比实验。对照组采用纯大模型单阶段排序方法，即不对候选进行任何预处理，直接将全部原始候选的描述提交给 GPT 3.5 Turbo 进行排序。实验组采用两阶段筛选方法，即先执行规则粗筛得到 22 个候选，再将这 22 个候选的描述提交给同一个大模型进行排序。两组实验均采用 Top 5 输出，即最终保留 5 个候选输入后续的导航模型。

表 5.4 展示了对比实验结果：

筛选方法	验证集 SR (%)	平均每次 API 输入候选数 ↓	相对 API 调用成本
纯大模型排序	39.71	45	1
两阶段筛选	39.67	22	0.48

表 5.4 两阶段筛选与纯大模型排序的对比

从表中可以看出，两阶段筛选方法在导航性能上与纯大模型排序方法几乎无差异：验证集成功率分别为 39.67%和 39.71%，差值小于 0.05 个百分点，可以认为在统计上无显著差异。然而，在大模型调用成本方面，两阶段方法带来了显著的优势。纯大模型排序方法每次导航步骤需要将 45 个候选的描述全部输入模型，每个候选描述的平均长度约为 50 个 token，单次 API 调用的输入 token 数约为 2250。两阶段方法中规则粗筛后的候选数量降至 22 个，输入 token 数降至约 1100。考虑到规则粗筛的执行时间可以忽略不计，两阶段方法将 API 调用成本降低了约 52%。

这一结果的原因在于：规则粗筛阶段移除的候选主要是位于页面边缘区域的非内容链接，例如版权声明、社交分享按钮、语言切换链接等，这些候选与导航指令的相关性极低。大模型在处理这些候选时几乎不会给予正面评分，因此将它们预先移除不会影响排序结果的准确性。两阶段设计使得大模型能够将计算资源集中于区分页面主体区域中语义相近的候选动作，从而在保持精度的同时大幅降低成本。这一特性使得本文提出的方法具备了实际工程部署的可行性。典型案例导航路径对比如图 5.5 所示：

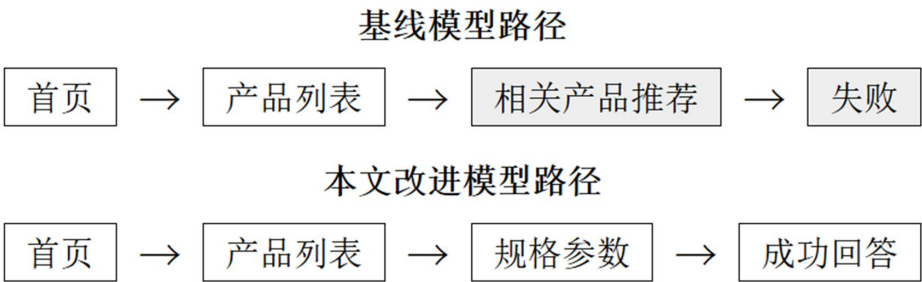


图 5.5 典型案例导航路径对比

5.6 典型案例分析与局限性讨论

本节通过具体案例展示候选筛选方法的效果，同时分析该方法存在的局限性。

成功案例一：产品规格参数查找任务。用户指令为“找到这款羊毛混纺袜子的规格参数，包括成分比例和洗涤说明”。基线模型的处理过程如下：第一步正确点击“产品”链接进入产品列表页；第二步面临多个候选链接，包括“产品详情”“相关产品推荐”“顾客评价”“规格参数”以及侧边栏的“隐私政策”“关于我们”。基线模型的注意力机制被“相关产品推荐”链接中的“推荐”一词吸引，错误点击该链接，进入了一个不相关的推荐页面，此后无法返回正确路径，最终因超过 10 步限制而失败。加入 Top5 筛选模块后，大模型对所有候选进行了语义相关性排序，输出结果为：第一位“规格参数”，第二位“产品详情”，第三位“顾客评价”，第四位“相关产品推荐”，第五位“关于我们”。该排序准确地将与指令最相关的“规格参数”置于首位。导航智能体接收到这一筛选后的候选集合后，直接选择排名第一的链接，一步到达目标页面，随后成功提取了规格参数信息并完成任务。

成功案例二：产品材质查询任务。用户指令为“告诉我这款毛衣的材质是什么”。目标页面上存在多个与材质相关的链接，分别为“Materials”“Material Care”“Materials Guide”和“Fabric Details”。基线模型在处理这些语义相近的候选时出现了混淆，选择了“Material Care”链接，进入了一个关于洗涤保养的页面，而该页面并不包含材质成分信息。加入筛选模块后，大模型不仅使用了候选自身的文本信息，还结合了每个候选的周围上下文文本。具体而言，“Materials”链接的父级容器标题为“Product Details”，而“Material Care”的父级容器标题为“Care Instructions”。大模型在排序时赋予了父级标题较高的权重，最终将“Materials”排在第一位。这一案例表明，本文设计的候选动作语义化重构方法通过加入位置描述和周围上下文文本，能够在一定程度上缓解纯文本信息的局限性。

局限性一：对视觉布局信息的利用不足。当前候选动作的语义化重构主要依赖文本元数据，包括链接文本、alt 属性和父级标题。然而在网页设计中，元素的视觉层次，例如字体大小、颜色、空间位置、背景色，往往隐含了该元素的重要

性。一个以大号红色字体显示的“立即购买”按钮显然比一个灰色小字体的“条款与条件”链接更为重要。当前的纯文本描述无法传递这一视觉信息。未来的改进方向可探索将页面截图区域的特征编码与文本描述相结合，形成多模态的候选表示，借鉴 CLIP^[6]和 Oscar^[13]等模型的图文对齐技术。

局限性二：对高度凝练技术术语的识别能力有限。在包含大量技术文档链接的复杂页面中，例如编程语言文档网站或学术论文网站，链接文本往往是高度简练的专业术语，如“API Reference”“Datasheet v2.3”“Errata v1.5”。大语言模型有时会将这些术语误判为无关内容，因为它们在通用语料库中出现的频率较低。在本文实验的一个案例中，正确的目标链接为“Errata v1.5”，大模型将其排在了第7位，认为其相关性低于其他更通用的链接。解决这一问题需要针对特定领域进行提示工程优化，或者在候选描述中加入该术语在页面中出现频次、位置凸显程度等统计特征。

局限性三：对动态加载内容的支持不足。现代网页大量使用 JavaScript 动态加载内容，初始 DOM 树中可能不包含所有可点击元素，或者元素的可见性状态会随着用户的滚动或点击行为而改变。本文采用的规则粗筛基于静态 DOM 属性进行判断，无法准确识别动态加载后才出现的元素，也无法判断某个元素在当前视口中是否真正可见。这可能导致部分有效的候选被误过滤，或者在动态内容丰富的单页应用中性能下降。解决这一问题需要引入浏览器自动化工具如 Selenium 或 Playwright，以实时获取页面的完整渲染状态和元素可见性信息。

上述局限性构成了本文方法的主要改进空间，也为后续研究提供了明确的方向。

5.7 实验结果总结

综合本章的全部实验分析，可以归纳出以下四项主要结论。

第一，基于大语言模型的候选动作筛选方法能够在保持原有 WebVLN 模型主体结构和参数不变的前提下，通过压缩决策空间并降低输入噪声，显著提升网页导

航任务的成功率与路径效率。在最优参数设置下，测试集成功率较基线提升约 0.9 个百分点，SPL 提升约 1.1 个百分点。

第二，参数 k 的选取对模型性能具有决定性影响。 k 值过小会导致正确动作保留率下降， k 值过大则残留过多噪声。本实验环境下 $k=5$ 能够在关键信息保留与噪声抑制之间取得最佳平衡。正确动作保留率与成功率之间的正相关关系从定量角度验证了输入质量对决策质量的直接影响。

第三，本文设计的两阶段筛选机制，规则粗筛与语义精排相结合，在保证筛选精度的前提下将大模型 API 调用成本降低约百分之五十。这一特性使得本文方法具备了实际工程部署的可行性，能够以可控的计算开销处理大规模的网页导航请求。

第四，通过对基线模型的系统复现和深入分析，本文确认了候选动作空间中的噪声干扰是原有模型决策效率低下的主要障碍。具体表现为模型倾向于点击导航栏或侧边栏的高频通用链接，而非与指令语义最匹配的深层内容链接。本文提出的候选筛选方法直接针对这一问题，实验证明该改进思路是有效的。

综上所述，本章完整呈现了从基线模型复现到改进方法设计再到系统性实验验证的全过程。实验结果有力地支持了本文的核心主张：将大语言模型作为语义过滤器来优化多模态决策模型的输入空间，是提升网页自动导航智能体性能的一条可行且有效的技术路径。

第 6 章 结论与展望

6.1 研究结论

本文围绕 WebVLN 任务中候选动作空间高噪声、大规模导致的决策效率低下问题，提出并验证了一种基于大语言模型的候选动作筛选优化方法。在成功复现基线模型并深入分析其决策机制的基础上，本文设计了一种可插拔的候选筛选框架，通过将结构化候选特征转化为语义化文本描述，利用大语言模型进行相关性排序

与 Top k 筛选，有效压缩了动作空间并降低了噪声干扰。实验结果表明，在适当的参数设置下，该方法在成功率、路径效率等关键指标上均取得了优于基线模型的表现，同时显著缩短了成功任务的导航路径长度。此外，所提出的两阶段筛选策略在保证筛选精度的前提下将大语言模型 API 调用成本降低约百分之五十，显示出良好的实用前景。本文的主要贡献可以概括为以下四点。

1. 成功复现了 WebVLN 基线模型，完整呈现了环境搭建、数据加载、训练流程与验证策略，并对原模型的失败模式进行了系统诊断，确认了候选噪声是制约性能的主要障碍。

2. 提出了一种基于大语言模型的候选动作语义排序与筛选方法，通过将候选重构为结构化文本描述并利用大模型的零样本语义匹配能力，实现了对动作空间的高质量压缩。

3. 设计了两阶段筛选机制，规则粗筛与语义精排相结合，在保持筛选精度的同时大幅降低了大模型的调用开销，增强了方法的工程可行性。

4. 通过多组对比实验，验证了参数 k 对性能的决定性影响，揭示了压缩率与保留率之间的权衡关系，并为后续研究提供了可量化的参考基准。

本文的工作为利用大语言模型优化多模态导航系统的输入质量提供了一种新的有效思路，也为后续研究在输入空间设计方面提供了可借鉴的范式。

6.2 研究局限

尽管本文方法取得了初步成效，但仍存在若干局限，这些局限既是当前方法的不足之处，也为后续研究指明了改进方向。

1. 对视觉布局信息的利用不足。当前候选动作的语义化重构主要依赖文本元数据，包括链接文本、alt 属性和父级标题。然而在网页设计中，元素的视觉层次，例如字体大小、颜色、空间位置、背景色，往往隐含了该元素的重要性。一个以大号红色字体显示的“立即购买”按钮显然比一个灰色小字体的“条款与条件”链接更为重要。当前的纯文本描述无法传递这一视觉信息。未来可探索将页

面截图区域的特征编码与大语言模型的多模态理解能力相结合，构建图文并茂的候选描述，使模型能够同时理解“这是什么”以及“它在视觉上有多重要”。

2. 对高度凝练技术术语的识别能力有限。在包含大量技术文档链接的复杂页面中，链接文本往往是高度简练的专业术语，例如“API Reference”“Datasheet v2.3”“Errata v1.5”。大语言模型有时会将这些术语误判为无关内容，因为它们在通用语料库中出现的频率较低。在本文实验的一个案例中，正确的目标链接为“Errata v1.5”，大模型将其排在了第7位，认为其相关性低于其他更通用的链接。解决这一问题需要针对特定领域进行提示工程优化，或者在候选描述中加入该术语在页面中出现频次、位置凸显程度等统计特征。

3. 对动态加载内容的支持不足。现代网页大量使用 JavaScript 动态加载内容，初始 DOM 树中可能不包含所有可点击元素，或者元素的可见性状态会随着用户的滚动或点击行为而改变。本文采用的规则粗筛基于静态 DOM 属性进行判断，无法准确识别动态加载后才出现的元素，也无法判断某个元素在当前视口中是否真正可见。这可能导致部分有效的候选被误过滤，或者在动态内容丰富的单页应用中性能下降。解决这一问题需要引入浏览器自动化工具如 Selenium 或 Playwright，以实时获取页面的完整渲染状态和元素可见性信息。

4. 当前筛选模块的参数 k 固定，未考虑不同导航阶段对候选数量的动态需求。在任务起始阶段，智能体可能需要保留较多候选以应对不确定性；而在任务后期，目标明确时应减少候选以加速收敛。固定 k 的设置无法适应这种阶段性变化。后续研究可引入自适应 k 值机制，例如根据当前候选得分分布的方差或模型预测置信度动态确定 k 的取值。

5. 本文工作目前将大语言模型筛选作为独立的前置步骤，未与 WebVLN 导航模型的训练过程进行联合优化。筛选模块生成的排序结果仅作为候选子集的选取依据，而导航模型在训练时并未感知到筛选过程的存在。未来可尝试将筛选模块的排序损失与导航模型的决策损失进行端到端联合训练，使导航模型能够主动适应筛选后的候选分布，有望进一步释放性能潜力。

6.3 未来展望

基于上述局限和对前沿研究的跟踪，本文认为以下几个方向值得在未来深入探索。

1. 多模态候选表示与筛选。将页面截图的局部区域特征与文本描述联合输入多模态大语言模型，使模型能够同时利用视觉和文本信息进行相关性判断。借鉴 CLIP^[6]、Oscar^[12]、Flamingo^[13]等模型的图文对齐技术，可以构建更加丰富的候选表示。例如，将候选元素的截图区域裁剪下来作为图像输入，与指令文本一起送入多模态大模型，让模型直接输出相关性得分。这一方法有望解决视觉布局信息利用不足的问题。

2. 自适应动态筛选机制。根据不同导航阶段的特征动态调整保留的候选数量。可以设计一个轻量级的元网络，输入当前状态和候选得分分布，输出最优的 k 值。也可以采用强化学习方法，将筛选动作视为策略的一部分，以导航成功率为奖励信号进行端到端优化。动态筛选机制有望在不确定性的起始阶段保留更多候选，在目标明确的后期阶段进一步压缩空间。

3. 筛选与导航的联合训练。将大语言模型的排序结果转化为可微分的软权重，使其能够参与导航模型的梯度回传。例如，将排序得分作为候选特征的软注意力权重，或者使用 Gumbel Softmax 技巧使筛选过程可微分。通过联合训练，筛选模块可以学习到更加有利于导航决策的排序策略，而非仅仅基于零样本的语义相似度。

4. 错误恢复与回溯机制。当前模型在导航过程中一旦选择错误动作，后续决策往往会沿着错误路径继续，难以自纠。未来可以引入基于大语言模型的错误检测与回溯模块，当模型检测到当前路径偏离指令目标时，能够主动回退到之前的状态并尝试其他候选。这一方向需要结合大模型的规划能力和网页导航的图结构信息，具有较高的研究价值。

5. 跨领域泛化能力研究。本文实验集中在电商网站的 seen 环境中，模型的跨领域泛化能力尚未充分验证。未来可以在更广泛的网页类型上进行测试，包括学

术论文网站、在线教育平台、政府服务门户等，评估筛选方法在不同领域间的迁移效果。同时可以探索基于领域自适应的提示工程方法，通过少量示例或指令微调使大模型快速适应新领域的术语和页面结构。

6.4 结束语

本文围绕网页自动导航任务中候选动作空间的噪声问题，提出了一种基于大语言模型的筛选优化方法，并通过系统的实验验证了其有效性。本文的工作为网页导航智能体的输入优化提供了一个切实可行的起点。随着多模态大模型技术的持续演进，更深层次地融合大模型的感知与推理能力包括子目标分解、错误恢复、闭环规划等将成为提升网页自动导航智能体水平的重要发展方向。希望本文能够为后续研究提供有价值的参考，并激发更多关于多模态决策系统输入质量提升的探索。

参考文献

- [1] Anderson P, Wu Q, Teney D, et al. Vision-and-Language Navigation: Interpreting visually-grounded navigation instructions in real environments[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2018: 3674-3683.
- [2] Tan H, Bansal M. LXMERT: Learning Cross-Modality Encoder Representations from Transformers[C]//Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing. 2019: 5100-5111.
- [3] Lu J, Batra D, Parikh D, et al. ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks[C]//Advances in Neural Information Processing Systems. 2019.
- [4] Chen Y C, Li L, Yu L, et al. UNITER: Learning Universal Image-Text Representations[C]//European Conference on Computer Vision. Springer, 2020: 104-120.
- [5] Devlin J, Chang M W, Lee K, et al. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding[C]//Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics. 2019: 4171-4186.
- [6] Radford A, Kim J W, Hallacy C, et al. Learning Transferable Visual Models From Natural Language Supervision[C]//International Conference on Machine Learning. PMLR, 2021: 8748-8763.

- [7] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need[C]//Advances in Neural Information Processing Systems. 2017: 5998-6008.
- [8] Chen Q, Pitawela D, Zhao C, et al. WebVLN: Vision-and-Language Navigation on Websites[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2024, 38(2): 1165-1173.
- [9] Hochreiter S, Schmidhuber J. Long Short-Term Memory[J]. Neural Computation, 1997, 9(8): 1735-1780.
- [10] Sutton R S, Barto A G. Reinforcement Learning: An Introduction[M]. MIT Press, 2018.
- [11] He K, Zhang X, Ren S, et al. Deep Residual Learning for Image Recognition[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 770-778.
- [12] Dosovitskiy A, et al. An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale[C]//International Conference on Learning Representations. 2021.
- [13] Li X, Yin X, Li C, et al. Oscar: Object-Semantics Aligned Pre-training for Vision-Language Tasks[C]//European Conference on Computer Vision. Springer, 2020: 121-137.
- [14] Furuta H, Lee K H, Nachum O, et al. Multimodal Web Navigation with Instruction-Finetuned Foundation Models[C]//International Conference on Learning Representations. 2024.
- [15] Lai H, Liu X, Liu J, et al. AutoWebGLM: Bootstrap And Reinforce A Large Language Model-based Web Navigating Agent[J]. arXiv preprint arXiv:2404.03648, 2024.
- [16] He H, Liao Y, Zhang J, et al. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models[C]//Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics. 2024: 6864-6890.
- [17] Liu J, Wang X, Lai H, et al. WEPO: Web Element Preference Optimization for LLM-based Web Navigation[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2025.
- [18] Chae H, Kim N, Yun S, et al. Web Agents with World Models: Learning and Leveraging Environment Dynamics in Web Navigation[J]. arXiv preprint arXiv:2410.13232, 2024.
- [19] Bhathal T, Sharma A, Patel R, et al. WebSight: A Vision-First Architecture for Robust Web Agents[J]. arXiv preprint arXiv:2508.16987, 2025.
- [20] Zhou G, Hong Y, Wu Q. NavGPT: Explicit Reasoning in Vision-and-Language Navigation with Large Language Models[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2024, 38(7): 7641-7649.

[21]Zhou G, Hong Y, Wang Z, et al. NavGPT-2: Unleashing Navigational Reasoning Capability for Large Vision-Language Models[C]//European Conference on Computer Vision. Springer, 2024: 260-278.

致谢

在本论文的完成过程中，我得到了许多老师和同学的帮助与支持，在此谨向所有给予我帮助的人表示衷心的感谢。首先，我要特别感谢我的指导教师许萌老师和黄赵老师。在论文选题、研究思路以及实验复现过程中，许老师和黄老师都给予了我耐心细致的指导。从最初的课题确定，到实验方案的设计，再到论文写作过程中的修改与完善，许老师和黄老师都提供了宝贵的建议，使我能够顺利完成本论文的研究工作。其次，感谢学院的各位任课教师。在本科阶段的学习过程中，各位老师为我系统地讲授了人工智能与计算机科学相关课程，使我掌握了扎实的理论基础，为本论文的研究奠定了重要基础。最后，我要感谢我的家人。在整个大学学习期间，家人一直给予我支持与鼓励，使我能够专心完成学业与毕业论文研究。谨以此论文向所有关心和帮助过我的老师、同学和家人表示诚挚的感谢。

2026 年 3 月